PennyPilot — Team Contribution & Kanban Evidence Report

Prepared by Mouni Sri Nallapaneni and Akshitha Sainath Sanagarapu

Project commencement: October 14, 2026 Proposal milestone: October 18, 2026 GitHub repository: https://github.com/MouniSri039/PennyPilot ClickUp board: https://app.clickup.com/90141780871/v/b/2kydwuw7-354

In this report, we explain how we planned and built PennyPilot together, from dividing the work to bringing everything together, testing it, and preparing the documentation. We have included the context behind our work, not just a list of GitHub commits.

1. Project overview and team approach

PennyPilot is an offline personal finance and savings application built with Flutter and Dart. Our goal was to create an application that helps a user understand everyday income and expenses, organize transactions, monitor budget usage, and work toward savings goals without depending on a cloud backend for its core functionality. We wanted the app to feel like one complete experience, not a set of separate screens. This meant that the interface, local data storage, state management, validation, reporting, and testing all needed to work together.

We started the project on October 14, 2026. During the initial stage, we discussed the expected user workflows, reviewed the project requirements, and divided the work so that both team members could own substantial implementation areas. October 18 was our proposal milestone. After that, we moved through application design and foundation work, core finance functionality, additional features, integration, testing, and final documentation. The detailed phase descriptions below represent our recollection of the project sequence. The GitHub history separately records the dates on which the visible commits were created; those commit dates should not be mistaken for the date on which every underlying idea or task was first discussed.

We each took responsibility for different features, but worked together to make sure they fit into the same app. We used GitHub branches and pull requests to bring changes into the main branch, and we used the ClickUp board to assign and track work. This helped us make progress on different parts of the app while checking that everything still worked well together.

2. Development timeline and milestones

October 14–17: kickoff, requirements, and planning. We began by aligning on the purpose of PennyPilot and the workflows that the application needed to support. We discussed the core finance experience, the need for local persistence, the main screens, and the responsibilities that could be developed in parallel. We also began organizing tasks so that implementation could be divided into manageable areas instead of treating the application as one large task.

October 18: proposal milestone. We treated October 18 as the proposal milestone for the project. The proposal established the direction for the application and provided a reference point for the work that followed. This date is part of the team's stated project chronology; it should be supported in the final submission with the authentic proposal document or dated course evidence.

October 19–21: application foundation and navigation. During the foundation stage, the project moved toward a working Flutter application with finance models and the core screen structure. The initial application setup and screen foundation are reflected in Akshitha's October 29 commit history. We distinguish that visible commit date from the broader planning and design period described by the team.

October 22–24: core finance workflows. The team's planned functional direction centered on recording income and expenses, maintaining categories, and allowing users to review their financial activity. The project task list also included local SQLite CRUD and navigation across the main application screens. The final application uses SQLite-backed local persistence, but the exact dates on which each part was first implemented should be verified against the full commit history and ClickUp activity before being stated as historical facts.

October 25–27: feature development and refinement. Work expanded beyond basic transaction entry into budget alerts, transaction search and filtering, sorting, CSV export, savings challenges, and financial insights. The two members took ownership of different feature areas and added tests alongside implementation. The detailed contribution sections below explain those areas and the commit evidence supplied for each member.

October 28–29: integration and refinement. We brought feature work together and continued improving code quality and application behavior. The visible GitHub history includes the initial PennyPilot Flutter app commit on October 29. We used integration and review to identify changes that needed to be made compatible with the shared application state and database structure.

October 30–31: final integration, testing, and submission evidence. The supplied GitHub history shows a substantial group of implementation, test, documentation, and evidence commits on these dates. Mouni's financial-intelligence and savings-challenge work is particularly visible in this period. Akshitha's transaction, budget-alert, CSV, testing, and AI-documentation work is also represented in the supplied history. The reported test result was 153 passing tests, zero failures, and 80.28% coverage.

This timeline explains the project from our team perspective. It does not claim that every feature was committed on the date of the phase in which it is discussed. For the final course evidence, dated ClickUp snapshots and task activity should be attached wherever available.

3. Mouni Sri Nallapaneni — individual contribution

My main contribution focused on the financial intelligence features and the savings-challenge experience. I worked on the service and state-management layers as well as the user-facing screens, because I wanted the features to be connected to the actual finance data rather than functioning as standalone demonstrations.

Financial intelligence and insights. I began by adding the financial intelligence service foundation and then connected that service to the Provider-based application state. This integration was important because the insights needed access to the user's stored transactions and current financial context. I added the financial insights dashboard and navigation so that the user could reach the analysis through the application. I then improved the time-aware financial analysis and its regression coverage, added historical forecast accuracy with SQLite persistence, and implemented explainable spending anomaly detection. These changes were intended to make the insights more useful and understandable, while keeping the analysis connected to persisted application data.

Savings-challenge workflows. I worked across the savings-challenge lifecycle rather than only adding a challenge screen. The work included stronger validation and progress feedback, editing existing challenges, and adding filtering and sorting so that users could manage multiple goals more easily. I added a progress-history model, the SQLite migration and database methods needed to persist that history, and the FinanceState integration that connects savings contributions to challenge progress. I also worked on making savings contributions atomic so that a contribution and the

corresponding progress update remain coordinated. Finally, I added the contribution-history UI so users can review how a challenge has progressed over time.

CSV export and testing. I developed the savings-challenge CSV export service and its tests, then integrated the export capability into the challenge workflow. I also added transaction-screen widget tests and performed cleanup related to the CSV export implementation. These tests helped validate the feature behavior through the UI and service layers rather than relying only on manual interaction.

Theme, state quality, and documentation. I added persistent dark-mode support and refined the theme configuration so the selected appearance would remain available across app restarts. I also made changes to improve state testability. Toward the end of the project, I contributed to finalizing the reported test coverage, adding application screenshots, and updating the README and architecture documentation. These tasks helped make the implementation easier to understand and the final project easier to review.

Visible GitHub evidence supplied for my work. The commit titles include “Add financial intelligence service foundation,” “Integrate financial intelligence with provider state,” “Add financial insights dashboard and navigation,” “Improve time-aware financial analysis and regression coverage,” “Add historical forecast accuracy and SQLite persistence,” and “Add explainable spending anomaly detection.” The savings work is represented by commits for validation, editing, filtering and sorting, CSV export, progress history, database integration, atomic contributions, and the contribution-history UI. The supplied history also includes persistent dark-mode, test coverage, screenshots, README, and architecture-documentation commits. Relevant PR numbers supplied in the history include PR #3, PR #5, PR #6, PR #11, #12, and #13. These titles and PR numbers are presented as supplied; commit hashes have not been added because they were not provided.

4. Akshitha Sainath Sanagarapu — individual contribution

Akshitha's work concentrated on the core application foundation and the transaction and budget-management experience. Her contributions helped establish the application's main finance workflows and made it easier for users to enter, find, review, and export their financial records.

Application foundation and transaction validation. Akshitha contributed the initial Flutter application foundation, finance models, and core screens. Her transaction work included strengthening the transaction form validation and

adding unit tests for that behavior. Validation is important in a finance application because incorrect or incomplete amounts and transaction details can undermine the reliability of later totals, charts, and insights. She also improved spending-advice conditional readability and refined the settings data-deletion confirmation flow and history-chart conditional safety.

Transaction discovery and organization. Akshitha implemented transaction search by description and category, then added tests for search and filtering behavior. She added separate income and expense filters and covered those filters with tests. She also implemented transaction sorting by date and amount and added tests for those sorting options. Together, these features make the transaction history more practical to use as the number of records grows: users can narrow the displayed data, distinguish income from expenses, and organize the results in a meaningful order.

Transaction CSV export and sharing. Akshitha developed the transaction CSV export generator and tests covering formatting and security. She then integrated CSV export with native sharing in the settings area. This work connected the export logic to an application workflow that users can access, instead of leaving it as an isolated generator. The related tests provide evidence for the generated content and its handling.

Budget alerts and dashboard feedback. Akshitha implemented budget-alert thresholds and spending-status logic, and then displayed contextual budget alerts on the dashboard. She added tests for threshold boundaries and edge cases, as well as dashboard tests for alerts and progress indicators. These changes help the dashboard communicate when spending is approaching or exceeding a budget, giving the user feedback at the point where it is most visible.

Integration tests and AI documentation. Akshitha contributed an Android navigation integration smoke test and graduate-helper evaluation work. She also prepared and pushed both AI_FEATURE.md and AI_Usage_Log.md. The supplied history specifically includes “Add final AI usage documentation” and “Refine AI usage log with team perspective.” This documentation work is part of her contribution and should be credited accordingly in the final report. She also added final test-coverage evidence and performed test/code-quality cleanup, including removal of a duplicate SQLite test dependency import and use of a const category fixture in transaction model tests.

Visible GitHub evidence supplied for her work. The supplied history includes “Initialize PennyPilot Flutter app with finance models and core screens,” transaction validation and search/filter/sort commits, CSV generation and native-sharing commits, budget-alert implementation and dashboard commits, integration smoke-test and graduate-helper evaluation, code-quality refinements, final test-coverage evidence, and the two AI-documentation commits. PR references supplied include PR #1, #2, #4, #7, #8, #9, #10, and #14. Merge commits are not counted as additional feature implementation.

5. How our contributions fit together

Although we owned different feature areas, PennyPilot required the work to connect through common models, persistent storage, and shared application state. The transaction and budget-alert experience depends on reliable transaction data. Financial insights also depend on that data, while savings challenges need their own persisted goal and contribution records. CSV export needs to read the appropriate records and present them in a form that can be shared. For that reason, our implementation work was not independent at the application level: changes to models, database methods, Provider state, and navigation had to remain compatible as the project grew.

Akshitha's work made the day-to-day transaction experience stronger through validation, search, filters, sorting, budget feedback, and transaction export. Mouni's work extended the application with financial intelligence and a deeper savings-challenge workflow, including progress history and challenge export. Both members contributed to testing and refinement. Akshitha's AI feature and usage documentation and Mouni's README and architecture updates also supported the final handoff and explanation of the application.

The result is a connected offline finance application with a dashboard, transaction entry and detail workflows, history and insights, settings, categories, and savings challenges. The core data is stored locally using SQLite. Provider/ChangeNotifier is used for application state, and SharedPreferences is used for preferences such as theme selection. We have not described a cloud synchronization service because the core application is designed to work offline.

6. ClickUp planning and task traceability

We used ClickUp as the shared place to organize responsibilities and track the work. The task information supplied during preparation shows assignments covering the final presentation, demo video and submission ZIP; the Kanban contribution evidence PDF; AI feature and usage documentation; README and architecture documentation; dashboard charts and budget alerts; SQLite income and expense CRUD; five-screen navigation; Provider state management; the offline smart spending helper; savings-challenge persistence; and unit/widget tests. The Kanban evidence task itself was assigned to both team members.

ClickUp assignments and GitHub commits tell us different things about the work, so we have kept them separate. For example, a task may have been assigned to one or both members while the visible commits show which member authored particular implementation or documentation changes. In this report, Akshitha is credited with preparing and pushing AI_FEATURE.md and AI_Usage_Log.md because that authorship was specifically confirmed and supported by the supplied commit titles.

For the final submission, we should connect each task to its real ClickUp ID and title, assignee, acceptance criteria, due date, dated comments or status history, corresponding commit or pull request, and the test or review result used to mark it Done. We have not invented task IDs, timestamps, historical comments, WIP limits, or board snapshots. Where the board history is available, those details should be inserted from the actual ClickUp record.

7. Testing, quality assurance, and limitations

The test result reported by the team was 153 tests passed, 0 failed, with 80.28% coverage. This provides evidence that the application was checked beyond a basic successful launch. The supplied commit history also shows test work alongside feature development: transaction validation tests, search/filter/sort tests, budget-alert threshold and dashboard tests, CSV formatting/security tests, savings-challenge tests, widget tests, and an Android navigation integration smoke test.

Our application uses Flutter and Dart for the mobile client, Provider/ChangeNotifier for state management, SQLite through the FinanceDatabase layer for local persistence, and SharedPreferences for user preferences and theme settings. The reported feature set includes transaction CRUD, category organization, budget monitoring and alerts, history search/filter/sort, CSV export, savings challenges and contribution history, financial insights, spending anomaly detection, and persistent dark mode.

We have tried to describe only the features we can actually show. A notification-preference toggle is not the same as scheduled operating-system notifications, so this report does not claim that notifications are scheduled. Similarly, CSV export is evidenced, but CSV import is not claimed. The final Flutter analysis reported no issues.

8. Team reflection and workflow improvement Working on

PennyPilot showed us that finishing individual screens is only one part of building a mobile app. Features that appear separate to a user often depend on the same data models, database relationships, state updates, and navigation structure. We had to consider how a transaction affects the dashboard, how budget thresholds are reflected in the UI, how savings contributions change challenge progress, and how the stored data is used by insights and export features. Testing these connections was an important part of making the application behave as one system.

Our use of branches and pull requests gave us a way to integrate work without treating every change as a direct edit to the shared main branch. The feature split also helped each member develop ownership in a substantial part of the project. Preparing our final evidence also showed us something we could do better: task tracking should be updated continuously, not reconstructed near the deadline. For our next project, we would write acceptance criteria before implementation, keep tasks small enough to review, link the relevant PR or commit while the task is active, record blockers and review decisions when they occur, and move work to Done only after the agreed behavior has been tested and the evidence is attached.

We would also preserve dated board snapshots at the requested checkpoints. Doing this during the project would make it easier to demonstrate how work moved through the board and how task ownership changed over time. We should report workflow improvements only when the board or project records show the change and its result; where there is no historical record, we should describe the proposed improvement as a lesson for future work rather than claim that it was

already measured.

9. Final evidence checklist

Before submitting, we need to make sure the evidence package includes the original records. This includes the authentic ClickUp board snapshots requested by the course (October 18 proposal, October 21 UI/navigation, October 24 functional checkpoint, and October 28 final board), if those snapshots exist. If a requested historical snapshot was not captured, we should disclose that limitation rather than recreate an image and present it as historical evidence. We should also attach actual ClickUp task IDs and activity, the configured board statuses and any WIP limits or Definition of Done, direct commit/PR links, the final test output, and the latest flutter analyze output.

The final report PDF should be saved in the repository, for example at docs/Kanban_Contribution_Evidence.pdf, and README.md should point to the exact path and include the ClickUp board URL. The same PDF should be included in each member's final ZIP. Before uploading, we should open the PDF locally, verify that it is readable offline, confirm that the repository and board links are accessible to the instructor, and check that the final ZIP contains the required project and submission files.

Evidence integrity statement: This report is based on the team chronology, task information, and GitHub commit titles shared during preparation. The commit titles and PR numbers are reproduced as supplied. ClickUp task IDs, dated board activity, historical snapshots,

commit hashes, and final analysis output have not been independently verified here and should be added only from the original records.

PennyPilot | Kanban Contribution Evidence

Project: PennyPilot - Project One (Flutter/Dart) **Team:** Akshitha Sainath Sanagarapu and Mouni Sri Nallapaneni

Evidence compiled: October 31, 2026 **Source:** ClickUp board screenshots and GitHub pull-request links

ClickUp board: https://app.clickup.com/90141780871/v/b/2kydwuw7-354
GitHub repository: https://github.com/MouniSri039/PennyPilot

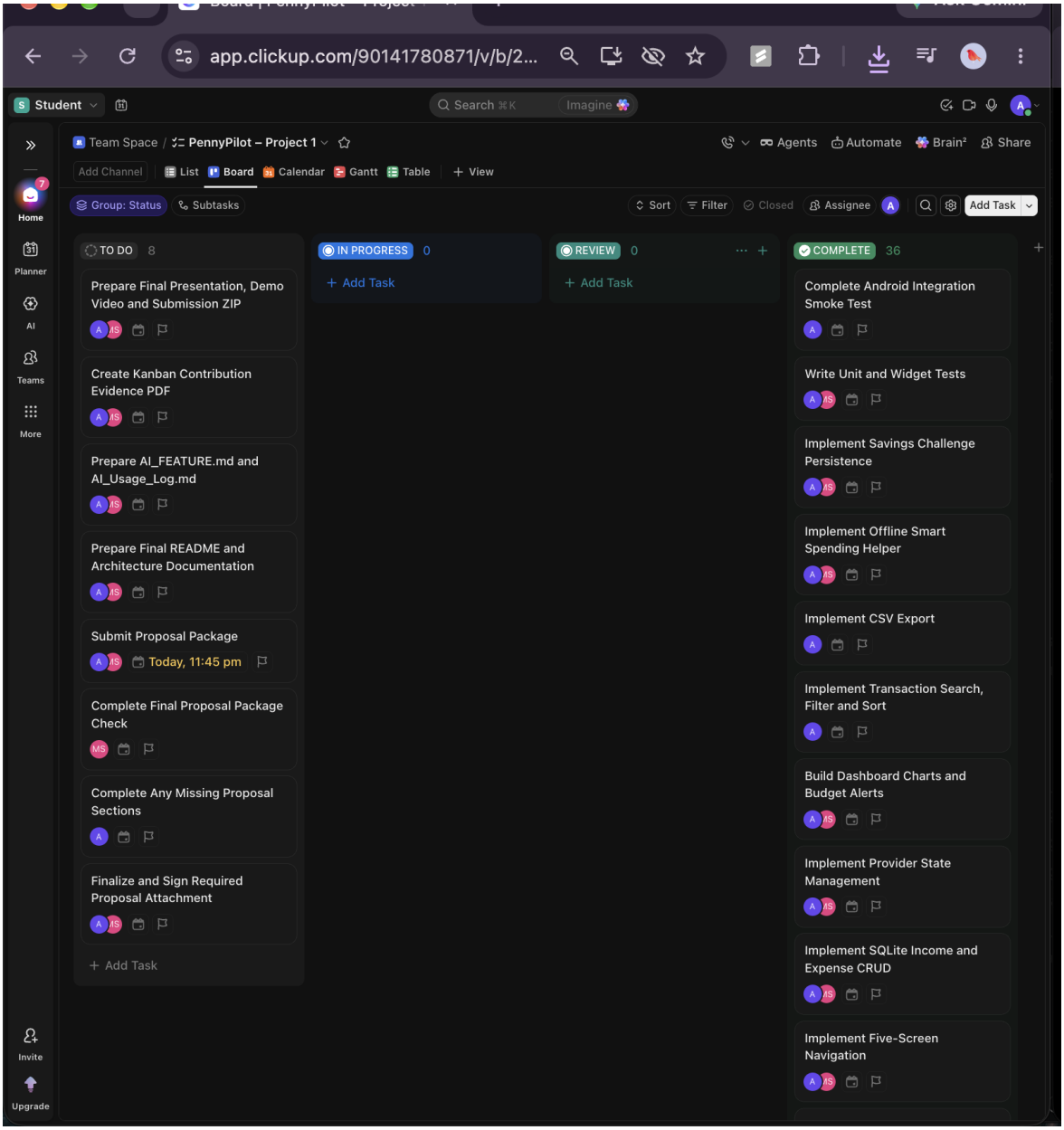
1. Board summary and workflow

The team used a ClickUp board with four workflow states: **To Do, In Progress, Review, Complete**. The board tracked proposal planning, implementation, testing, and final submission activities. The screenshot captured October 31 shows **36 Complete, 8 To Do, 0 In Progress, and 0 Review**. The eight To Do tasks include four outstanding proposal-related tasks and four final documentation/submission tasks.

Evidence limitation: The board was initially organized earlier in the project and comprehensively updated on October 31. These screenshots show the board as it existed when captured; they are **not** four separate historical daily snapshots. Historical progress is better corroborated by dated GitHub commits and pull requests.

2. Current board snapshot — October 31, 2026

The board below shows the task status totals and named implementation/testing tasks.


app.clickup.com/90141780871/v/b/2...
Student
Search ⌘K
Imagine
Team Space / PennyPilot – Project 1
Agents
Automate
Brain²
Share
Add Channel
List
Board
Calendar
Gantt
Table
View
Group: Status
Subtasks
Sort
Filter
Closed
Assignee
Add Task
Home
Planner
AI
Teams
More
Invite
Upgrade
TO DO 8
Prepare Final Presentation, Demo Video and Submission ZIP
Create Kanban Contribution Evidence PDF
Prepare AI_FEATURE.md and AI_Usage_Log.md
Prepare Final README and Architecture Documentation
Submit Proposal Package
Today, 11:45 pm
Complete Final Proposal Package Check
Complete Any Missing Proposal Sections
Finalize and Sign Required Proposal Attachment
Add Task
IN PROGRESS 0
Add Task
REVIEW 0
Add Task
COMPLETE 36
Complete Android Integration Smoke Test
Write Unit and Widget Tests
Implement Savings Challenge Persistence
Implement Offline Smart Spending Helper
Implement CSV Export
Implement Transaction Search, Filter and Sort
Build Dashboard Charts and Budget Alerts
Implement Provider State Management
Implement SQLite Income and Expense CRUD
Implement Five-Screen Navigation

3. Development and contribution trace

Workstream / ClickUp task	Contributor shown	Evidence / status
Android integration smoke test	Akshitha	PR #14: merged; approved review
Dashboard budget alert widget tests	Akshitha; test task shared	PR #10: merged
Transaction search, filter, sort	Akshitha	Completed task on board
CSV export	Akshitha	Completed task on board
Five-screen navigation	Akshitha + Mouni	Completed task on board
SQLite income/expense CRUD	Akshitha + Mouni	Completed task on board
Provider state management	Akshitha + Mouni	Completed task on board
Dashboard charts and budget alerts	Akshitha + Mouni	Completed task on board
Savings challenge persistence	Akshitha + Mouni	Completed task on board
Offline smart spending helper	Akshitha + Mouni	Completed task on board

Contributor labels above reproduce current ClickUp task assignments; shared assignments alone do not establish equal implementation ownership. GitHub history is the stronger source for individual authorship.

4. Verified GitHub review and testing evidence

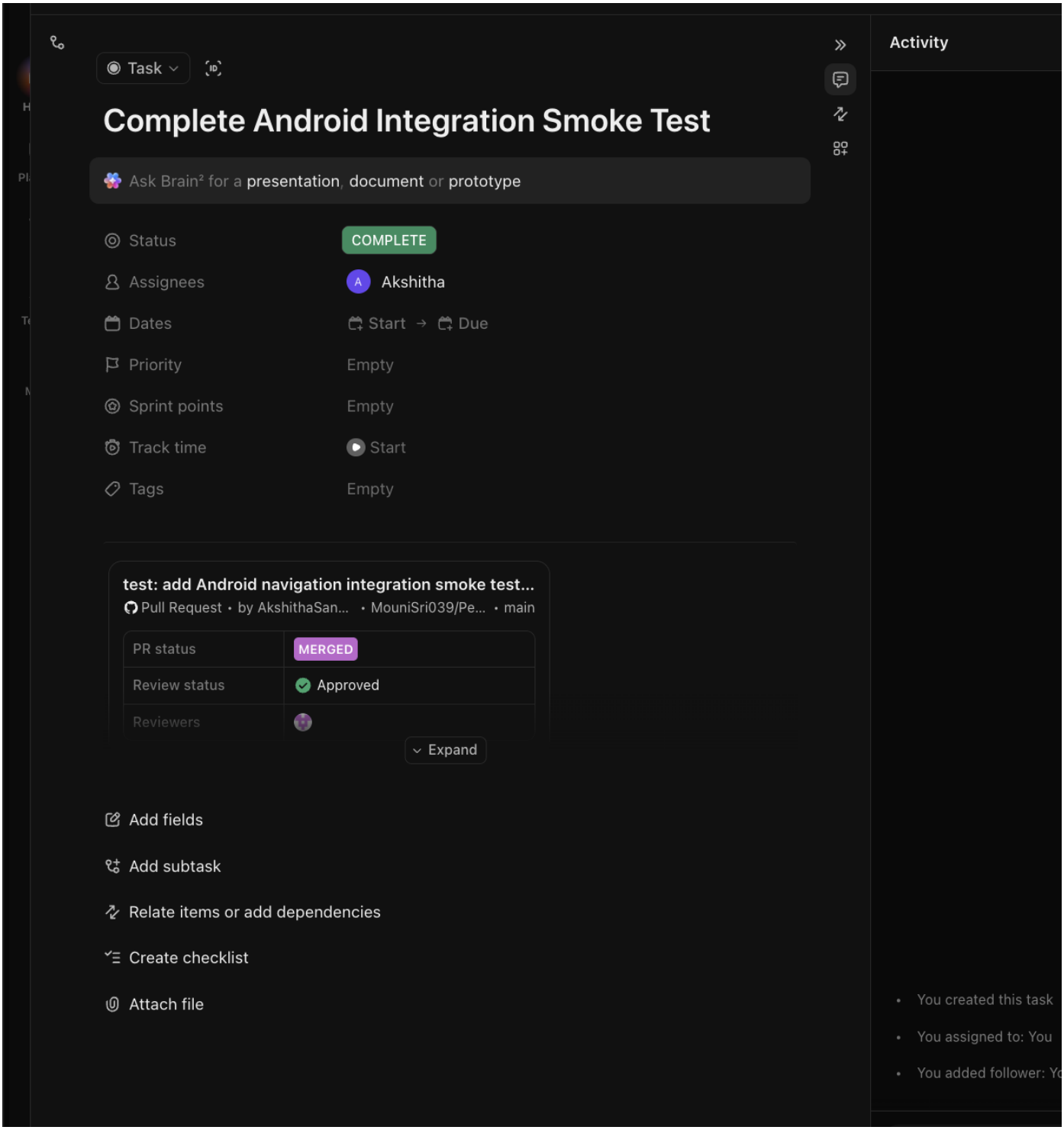
PR #14: https://github.com/MouniSri039/PennyPilot/pull/14 — Android navigation integration smoke test and spending-helper held-out evaluation test. ClickUp screenshot displays **MERGED** and **Approved**.

PR #10: https://github.com/MouniSri039/PennyPilot/pull/10 — Dashboard budget alert widget tests. ClickUp screenshot displays **MERGED**.

Verification at last recorded main-branch check: Flutter analyze reported no issues; Flutter test reported 107 passing tests. This is a recorded local verification result, not a new independent execution for this PDF. Coverage above 80% and final release APK were not yet confirmed.

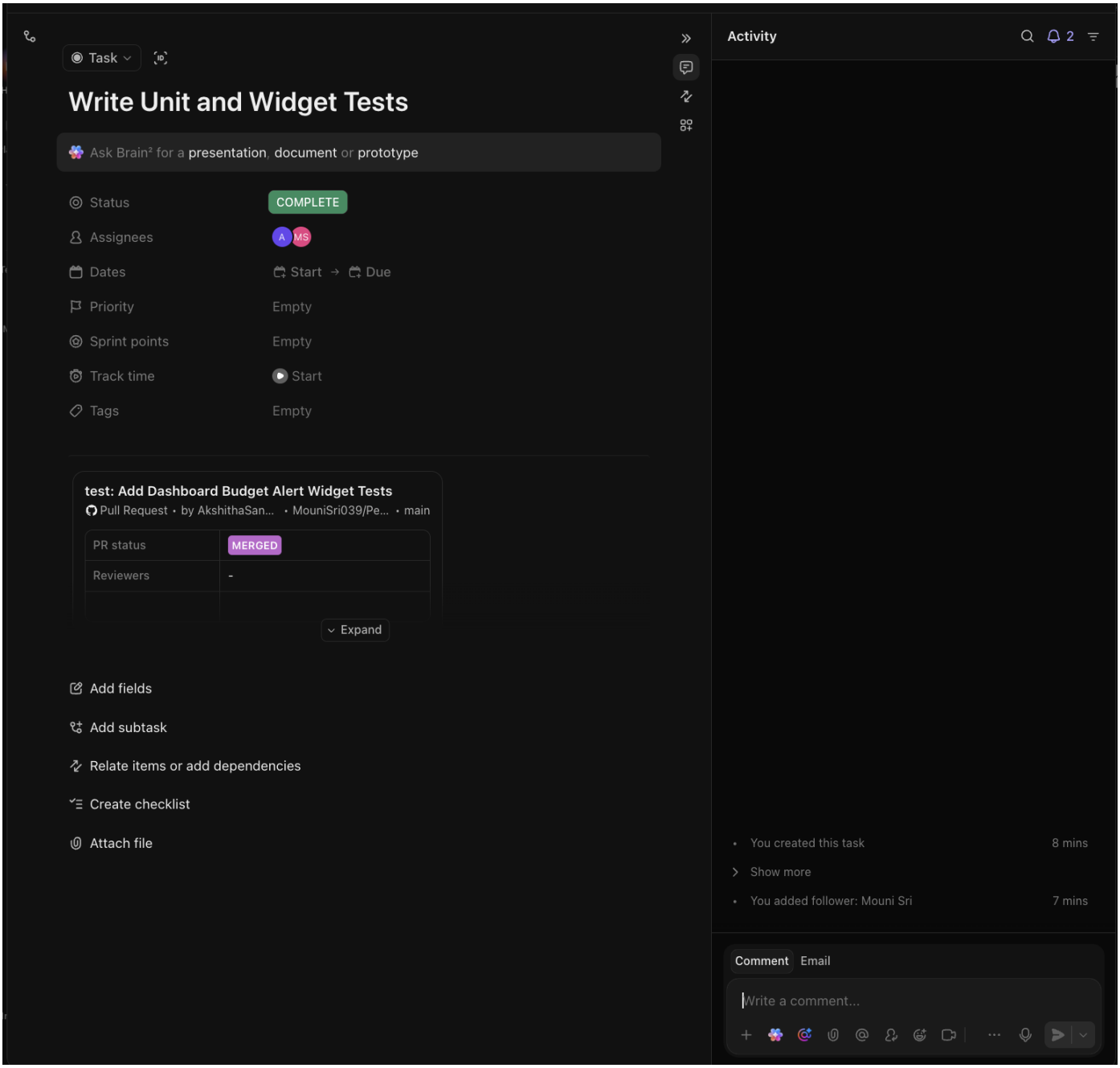
5. ClickUp task-to-PR linkage: integration test

Screenshot of the completed ClickUp integration smoke-test task with its linked GitHub pull request and approved merged status.



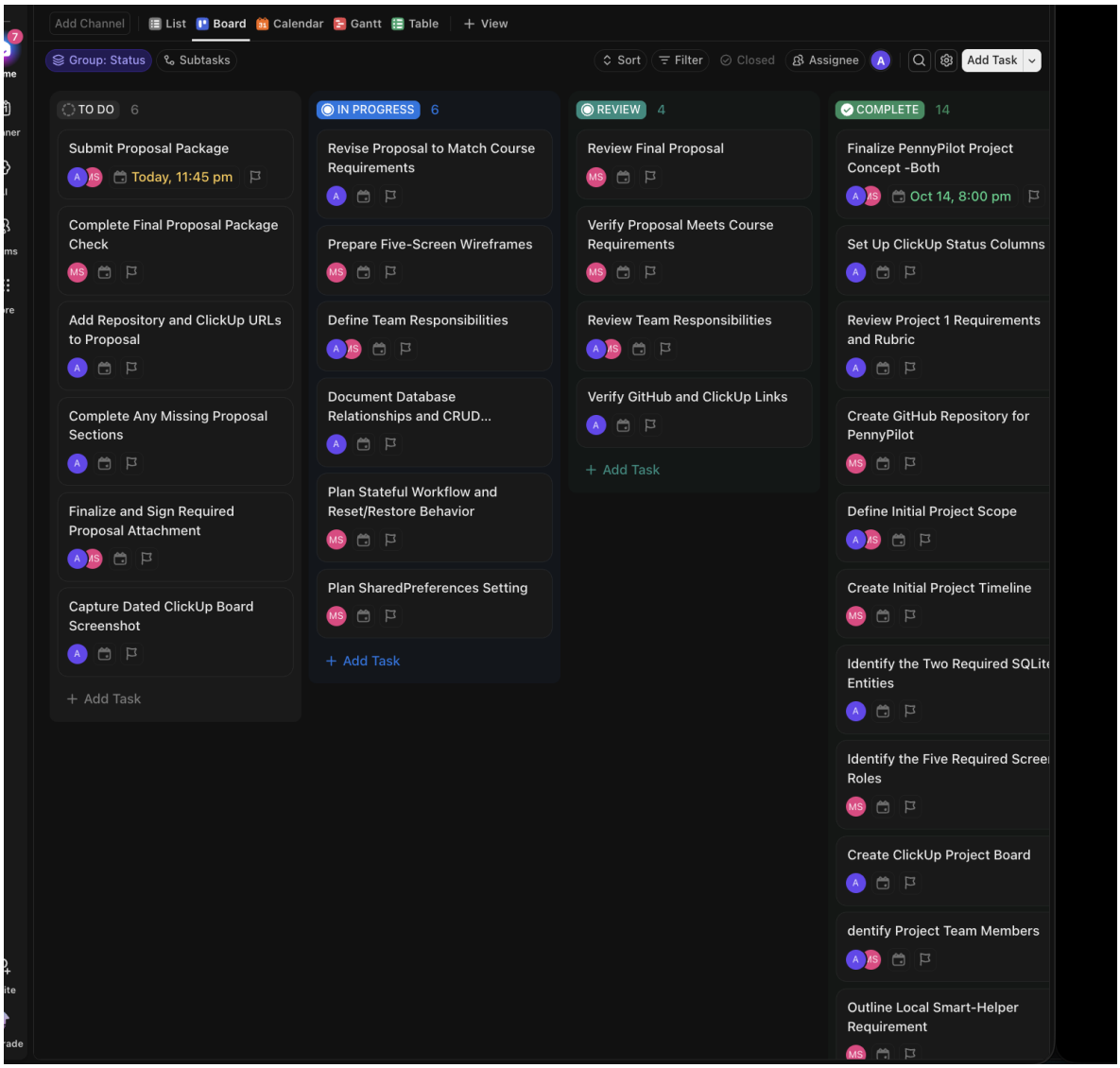
6. ClickUp task-to-PR linkage: widget tests

Screenshot of the completed ClickUp unit/widget testing task showing the merged dashboard-budget-alert widget-test pull request.



7. Board organization and outstanding work

Earlier October 31 board screenshot, before the final status cleanup and addition of development tasks. This is an earlier **same-day** snapshot, not evidence of activity on a different date.



Outstanding tasks at the time of the final screenshot

Final README/architecture documentation; AI_FEATURE.md and AI_Usage_Log.md; Kanban contribution evidence PDF; final presentation, demo video and submission ZIP; and four proposal-package tasks. This PDF addresses the Kanban evidence document task but does not prove it has been uploaded to the repository.

8. Collaboration practice and limitations

The project used named task assignees, shared work items, GitHub pull requests and review, and a four-column Kanban workflow. An improvement for future work is to update task statuses at each merge and save dated board snapshots during the project rather than retrospectively organizing the board at submission time.

Snapshot disclosure: Four distinct historical dated board screenshots are not available in the provided evidence. The screenshots included here are authentic captures from October 31, 2026. No earlier screenshots, activity dates, or task histories have been invented.